# REPLAYABLE MINIMUM-DISTANCE CERTIFICATES FOR STABILIZER CODES, WITH NO SOLVER AND NO PROOF ASSISTANT IN THE TRUSTED BASE: THE BIVARIATE-BICYCLE FAMILY THROUGH $n = 288$

DANIEL KIRTCHAKOV

*Date*: August 4, 2026.

2020 *Mathematics Subject Classification.* Primary 81P73; Secondary 94B05, 68V15, 03B35.

*Key words and phrases.* Quantum error correction, minimum distance, stabilizer codes, bivariate bicycle codes, gross code, LRAT, proof certificates, SAT.

*Computation and authorship.* All searches, encodings, symmetry-breaking constructions, and certificate designs in this work were produced by **Claude Fable 5** (Anthropic), directed by the author, on a single Apple M4 laptop (10 cores, 16 GB). The three independent checkers share no code with the generating pipeline, import only the Python standard library, and were exercised against the corpus by a separate agent instance that was given no access to the pipeline. This is a factual methods statement, and it is part of the point of the paper: the artifacts are designed so that the provenance of the *search* is irrelevant to the validity of the *result*. External tools used by the pipeline only: CaDiCaL 3.0.1 (`--lrat --no-binary`), Python 3.

*Prior-art record.* The credit statements in §1.4 are the output of a full-text adversarial sweep completed on August 4, 2026, in which every source was read in raw form (arXiv LaTeX source, GitHub repository trees at a named commit, package documentation) and in which two claims of an earlier internal draft were found to be wrong and withdrawn. The dated record of that sweep, listing what was read and what changed, ships with the artifacts as `SWEEP-RECORD-QEC-2026-08-04.md`. *Draft of August 4, 2026.*

Abstract. This is a verification contribution, not a discovery. The minimum distances of the codes treated here are already in the literature; what did not exist is a standalone artifact that a third party can replay without a SAT solver and without a proof assistant. We supply one for eleven stabilizer codes, together with three short checkers, and we submit the result to an independent audit: 47 certificate checks run, 47 passed, 0 failed; 5.08 GiB of LRAT replayed in pure Python inside a 79 MB peak resident set; all five bivariate-bicycle parity-check matrices rebuilt *byte-identically* from the published construction of Bravyi et al.; 182 of 182 manifest SHA-256 entries matching; 11 of 11 negative controls correctly rejected.

The formats are deliberately small. Upper bounds are witness pairs verified by three $\mathbb{F}_2$ matrix–vector products. Lower bounds are LRAT unsatisfiability proofs for a canonical CNF encoding of "some nontrivial logical has weight at most $K$", in which the checker regenerates the CNF from the raw parity-check matrices and machine-checks the three algebraic side conditions that make the encoding *exact*; neither the solver nor the shipped CNF is trusted, and for every code in the corpus, including the largest proof, replay needs nothing beyond the Python standard library.

For the gross code $[[144, 12, 12]]$ of Bravyi et al. we certify $d = 12$ end to end: weight-12 $X$- and $Z$-witnesses, and in each sector a single *symmetry-free* LRAT proof (868 MB for $X$, 672 MB for $Z$) of $d_X, d_Z \geq 12$, so that no symmetry lemma enters the trusted base for this code. For $[[288, 12, 18]]$ we certify $d_X \geq 14$ by a 2.94 GB LRAT proof, replayed in pure Python in 414 s. Bravyi et al. assert $d = 18$ for that code exactly, by integer programming and without a checkable artifact; the strongest lower bound previously *reported* by a certifying-capable method is $d \geq 11$, solver-asserted by Chen, Jafari and Lai with no proof files in their repository. Our $d_X \geq 14$ improves on that bound and is, as far as our sweep could determine, the only machine-checkable lower bound on record for this code. It is not evidence against $d = 18$.

We state the trusted base explicitly, separating what is machine-checked per certificate from the facts that are assumed, and we report the four defects the audit found, including one latent soundness hole in a checker branch. We claim priority neither for the distance values, which are Bravyi et al.'s, nor for machine-checked quantum distance proofs, which are LEAN-QEC's and whose public repository reports a completed gross-code verification as of 2026-07-10. What is offered here is a certificate format and a trusted base: a 649-line standard-library reader, and artifacts that outlive the tools that produced them.

## 1. Introduction

1.1. **The problem.** The minimum distance $d$ of a quantum error-correcting code is the single number that determines how many errors it can detect. It is also expensive: computing it is a minimum-weight problem over a coset space, intractable in general [Var97], and the standard tools for it in the quantum literature are of two kinds. Randomized searches such as QDistRnd [PSKK22] and Stim's `search_for_undetectable_logical_errors` [Gid21] return upper bounds and say so plainly; Stim's documentation is explicit that "this is a heuristic method". Exact methods — integer programming [LAR11], SAT and MaxSAT [CJL26], branch and bound — return a number and an implicit assertion that the search was exhaustive. The survey of Webster, Jacob and Higgott [WJH26] classifies the landscape carefully; what almost none of it produces is an artifact a third party can check.

This matters more than usual here, because the published distances of the codes now being built into hardware are load-bearing engineering constants. The bivariate-bicycle (BB) codes of Bravyi, Cross, Gambetta, Maslov, Rall and Yoder [BCGMRY] — in particular the $[[144, 12, 12]]$ "gross code" — are the basis of a fault-tolerant memory architecture. Their distances were computed by the mixed-integer-programming method of Landahl, Anderson and Rice [LAR11]; the paper is explicit about this and about which entries of its Table 3 are upper bounds

only. The gross-code value has since been confirmed independently, again by mathematical programming: Cruz-Benito, Cross, Kremer and Faro [CCKF26] close the MILP to a gap of zero and obtain $d = 12$ exactly. Nothing is wrong with either computation. But a reader who wants to check them must re-run a solver and trust the re-run; neither emits an object that survives the solver.

1.2. **What this note provides.** An artifact-first alternative. For each code we ship files that a skeptic can replay in isolation, and three short Python programs that replay them. The design constraints were:

(i) *No solver in the trusted base.* The checker never runs a SAT solver and never reads the CNF file the solver saw. It *regenerates* the CNF from the raw parity-check matrices, then replays the LRAT proof against its own clause list. A corrupted shipped CNF changes nothing — this was tested (§7).

(ii) *No proof assistant in the trusted base.* No Lean, no Mathlib, no toolchain fetch. The reader needs CPython and the files.

(iii) *The encoding's exactness is machine-checked, not asserted.* The reduction from "$d_X \geq K+1$" to "this CNF is unsatisfiable" is valid only under three algebraic side conditions on $(H_X, H_Z, P)$. The checker verifies all three, per certificate, over the raw matrices (Theorem 3.1).

(iv) *Everything the checker assumes is written down.* Section 4 is the honest inventory, including the items argued in prose and not machine-verified anywhere.

1.3. **Results.** Table 1 is the corpus. Every entry traces to a file in `certificates/`; the byte counts are the actual proof sizes and the replay times are wall clock from the independent audit of §7, measured with `/usr/bin/python3` (CPython 3.9.6, no `numpy`, no compiled helper of any kind).

Three things in that table deserve to be stated precisely, because each of them touches published work.

**(A) The gross code.** We certify $d([[144, 12, 12]]) = 12$ end to end. The upper bound is a pair of weight-12 witnesses, 970-byte JSON files, checked in 7 ms each. The lower bound is certified twice in each direction: once by a symmetry-broken pair of instances in the $X$ sector, and once, in *each* sector, by a single LRAT proof with no symmetry hypothesis at all — 867,803,294 bytes for $X$ and 671,988,205 bytes for $Z$. The symmetry-free pair is the one that matters for the trusted base: it removes the orbit lemma of §3.3 from the argument entirely, so $d([[144, 12, 12]]) = 12$ rests on nothing but the exactness theorem, the LRAT semantics, and the standard cardinality and XOR encodings.

We do *not* claim to be first to a machine-checked distance for this code. The value $d = 12$ is Bravyi et al.'s [BCGMRY, Table 3], confirmed exactly at MIP gap zero by Cruz-Benito, Cross, Kremer and Faro [CCKF26] and reproduced by SAT in [CJL26]. The idea of a kernel-checked quantum distance proof is LEAN-QEC's [ELWT26], and their public repository reports the gross code completed; see §1.4 for the dated record. What is ours is the certificate format, the symmetry-free variant, and the trusted base.

**(B)** $[[288, 12, 18]]$**.** We certify $d_X \geq 14$ by a 2.94 GB LRAT proof and $d_X \leq 18$ by a weight-18 witness. The value is not in dispute and we are not disputing it: Bravyi et al. assert $d = 18$ for this code exactly, by integer programming, and their Table 3 marks it without the "$\leq$" that flags their upper-bound-only entries (Remark 6.1, which corrects a misreading an earlier draft of our results file contained). What they do not supply — and what nobody supplies — is an artifact. The strongest lower bound previously *reported* by a method capable in principle of certifying is $d \geq 11$, from Chen, Jafari and Lai [CJL26] at a 7200 s timeout, solver-asserted,

| code | $n, k$ | certified | lower-bd. proof (bytes) | solver (s) | pure-Python replay (s) |
|---|---|---|---|---|---|
| Steane $[[7,1,3]]$ | $7,1$ | $d=3$ | 699 \| 699 | 0.02 \| 0.07 | 0.0003 \| 0.0009 |
| five-qubit $[[5,1,3]]$ | $5,1$ | $d=3$ | 1,514 | 0.02 | 0.0004 |
| rot. surface $d$=3 | $9,1$ | $d=3$ | 518 \| 746 | 0.03 \| 0.02 | 0.0004 \| 0.0004 |
| rot. surface $d$=5 | $25,1$ | $d=5$ | 4,685 \| 7,085 | 0.02 \| 0.02 | 0.001 \| 0.001 |
| rot. surface $d$=7 | $49,1$ | $d=7$ | 24,939 \| 75,107 | 0.02 \| 0.02 | 0.005 \| 0.009 |
| Golay $[[23,1,7]]$ | $23,1$ | $d=7$ | 226,377 each | 0.04 \| 0.04 | 0.024 \| 0.024 |
| BB $[[72,12,6]]$ | $72,12$ | $d=6$ | 1.54 \| 1.93 MB | 0.26 \| 0.30 | 0.14 \| 0.17 |
| BB $[[90,8,10]]$ | $90,8$ | $d=10$ | 128 \| 151 MB | 48.4 \| 50.8 | 10.9 \| 13.0 |
| BB $[[108,8,10]]$ | $108,8$ | $d=10$ | 71.6 \| 82.3 MB | 28.2 \| 24.7 | 6.2 \| 7.1 |
| **BB** $[[144,12,12]]$ | $144,12$ | $\mathbf{d=12}$ | 868 \| 672 MB$^\dagger$ | 342 \| 227 | 176 \| 72.9 |
| BB $[[288,12,18]]$ | $288,12$ | $14 \le d \le 18$ | 2.94 GB$^\ddagger$ | 513 | 414 |

TABLE 1. The certificate corpus. Paired entries are $X \mid Z$ sector. Solver times are CaDiCaL 3.0.1 on one Apple M4 laptop, recorded in each code's `meta.json`. Replay times are from the independent audit (§7) and use nothing outside the Python standard library. $^\dagger$Symmetry-*free* single-instance proofs; a symmetry-broken $X$ certificate (124 MB + 34 kB, solver 45.0 s) is also shipped and replays in 10.1 s. $^\ddagger$Symmetry-broken, $K = 13$, two instances (2,941,958,076 + 191,479 bytes); the ladder rungs $d_X \ge 10$ (65 MB) and $d_X \ge 12$ (351 MB) are shipped as well. At $n = 288$ the certified quantity is $d_X$; the passage to $d$ uses the shipped duality certificate together with $d = \min(d_X, d_Z)$, and that certificate was generated after the audit closed (Remark 3.5).

with no proof files in their repository. Our $d_X \ge 14$ improves on that bound and is, as far as the sweep of §1.4 could determine, the only machine-checkable lower bound on record for this code. It falls four short of 18, and that shortfall is a limitation of our encoding, not evidence.

**(C) The rest of the family.** The values $d = 6, 10, 10$ for $[[72,12,6]]$, $[[90,8,10]]$, $[[108,8,10]]$, and the distances of Steane, Golay, the rotated surface codes and the five-qubit code, are all prior art; several are computed exactly, without certificates, in [CJL26]. For Steane, Golay, $[[72,12,6]]$ and $[[90,8,10]]$, replayable proof artifacts also exist in the LEAN-QEC repository, so "certified" is shared ground there and our contribution for those four is the trusted base, not primacy. We include them because a certificate format is worth nothing without a regression suite, and because six of them admit an independent brute-force cross-check (§7.2).

1.4. **Provenance and credit.** This subsection was written after a first-hand check of the prior art on August 4, 2026, in the course of which two claims in an earlier internal draft of these results turned out to be wrong. What follows is the corrected record. Table 2 is the claim-by-claim version.

*The codes.* The bivariate-bicycle family, including $[[144,12,12]]$, is Bravyi, Cross, Gambetta, Maslov, Rall and Yoder's [BCGMRY]. All five parity-check matrices in our corpus were reconstructed from the construction in that paper and are byte-identical to it (§7.1). Their distances were computed there by the mixed-integer-programming method of Landahl, Anderson and Rice [LAR11]. Every distance value we certify for a BB code, and every distance value we certify for Steane, Golay, the surface codes and the five-qubit code, was already known. This note contributes *certificates for known numbers*, plus one number ($d_X \ge 14$ for $n = 288$) that is new only in the sense of being machine-checkable.

*Machine-checked quantum distance proofs.* The approach is LEAN-QEC's [ELWT26] (Ehatamm, Lee, Wu and Tao, 15 May 2026): a SAT encoding of the distance problem whose refutation is replayed inside the Lean 4 kernel. Their paper is candid about the gross code: it reports $[[144, 12, 12]]$ as dispatched by an external `cvc5` call rather than in the kernel, states that the 144-qubit instance pushes their certificate-replay budget beyond what they can verify in the kernel, and names kernel-side replay at that size as their next concrete engineering target. **Their repository has since moved past the paper.** At commit `c73827d` of `VerifiedQC/Lean-QEC`, dated 2026-07-10, a notes file reports that the $[[144, 12, 12]]$ BB code distance has been fully verified with `bv_decide` in about thirty minutes, by a pipeline whose final step translates the LRAT proof of unsatisfiability back into Lean and checks it with the kernel. We therefore make *no* priority claim for a machine-checked gross-code distance, and we have removed such a claim wherever it appeared in our own working notes.

Several differences remain, and we state them as observations of a moving repository at a single commit, which may already be stale. At `c73827d`, the file `BB144.lean` contains two `sorry` placeholders — `BB144_X_ker_rank` at line 69 and `BB144_Z_ker_rank` at line 72 — through which the end-to-end theorem `BB144_dist_12` routes, so that theorem is not `sorry`-free as committed; three further lemmas use `native_decide`, which their own paper notes extends the trusted base with Lean's compiler; and no LRAT artifact is committed for the 144-qubit instance, though LRATs are committed for smaller codes. Their 144-qubit encoding uses sorted-location symmetry constraints, i.e. it is symmetry-broken; we found no counterpart anywhere to the symmetry-free proofs of §5. We note also, since it would be easy to overstate their coverage in the other direction, that `BB108.lean` at the same commit carries `sorry` at lines 120 and 132 with its `bv_decide` invocation commented out, so their kernel-checked ladder should not be described as reaching $n = 108$ either. None of this is a criticism of work in progress; a repository ahead of its paper is a good problem to have. It is the reason we describe our contribution as a trusted base rather than a first, and we have made these statements as specific as we can so that they can be checked and, when they go stale, corrected.

*Mathematical programming, independently.* Cruz-Benito, Cross, Kremer and Faro [CCKF26] (1 June 2026, IBM) revisit the gross-code distance with a mixed-integer linear program and close it to a MIP gap of zero, confirming $d = 12$ exactly. That is a stronger form of the same kind of evidence as [BCGMRY]: an exhaustive search whose exhaustiveness is attested by the solver's own optimality certificate rather than exported as a checkable object. We cite it as the current reference point for the value, and note that the gap it leaves — a MILP optimality proof is not a file — is precisely the one this note fills.

*SAT-based distance computation at scale.* Chen, Jafari and Lai [CJL26] (29 May 2026) compute BB distances with a battery of solvers, obtaining $d = 12$ for $[[144, 12, 12]]$ exactly and, for $[[288, 12, 18]]$, lower bounds of 11 under a 7200 s budget. Their results are solver-asserted — their repository `guluchen/QDistSAT` contains no proof or certificate files — and their $d \geq 11$ is the strongest quantity previously published for that code *as a lower bound*, as against the exact value $d = 18$ asserted in [BCGMRY]. Our $d_X \geq 14$ improves on it and, for the first time, backs a bound for this code with a replayable artifact.

*Infrastructure.* The LRAT format and the checking discipline are due to Cruz-Filipe, Heule, Hunt, Kaufmann and Schneider-Kamp [CFHKS17], building on `drat-trim` [HHW13]; our internal checker implements RUP-with-hints against that format. The solver is CaDiCaL [Bie20]. The cardinality encoding is Sinz's sequential counter [Sin05]; the XOR gates are Tseitin's [Tse68]. The CSS structure is Calderbank–Shor [CS96] and Steane [Ste96]; the stabilizer formalism is Gottesman's [Got97].

| Item | Earliest source we verified | Status here |
|---|---|---|
| The BB codes; $d = 12$ for $[[144, 12, 12]]$; $d = 18$ for $[[288, 12, 18]]$ | Bravyi et al. [BCGMRY], Table 3 (MIP method of [LAR11]) | Re-verified, not claimed |
| $d = 12$ for $[[144, 12, 12]]$ confirmed exactly by MILP, MIP gap 0 | Cruz-Benito, Cross, Kremer, Faro [CCKF26], 1 June 2026 | Re-verified, not claimed; no certificate emitted there |
| $d_X = d_Z$ for BB codes | Bravyi et al. [BCGMRY], supplemental lemma | Cited; only the explicit permutation certificate is ours |
| Kernel-checked SAT distance proofs for quantum codes | LEAN-QEC [ELWT26], 15 May 2026 | Not claimed |
| Machine-checked $[[144, 12, 12]]$ distance | LEAN-QEC repository, commit `c73827d`, 2026-07-10 | Not claimed |
| $d$ computed exactly by SAT for $n \leq 144$ BB codes | Chen–Jafari–Lai [CJL26], 29 May 2026 | Re-verified, not claimed |
| $d \geq 11$ for $[[288, 12, 18]]$ by SAT | Chen–Jafari–Lai [CJL26] | Improved to $d_X \geq 14$, with artifact |
| *Offered as new:* The certificate format of §3 and its machine-checked exactness conditions; the symmetry-free $[[144, 12, 12]]$ proofs; the only machine-checkable lower bound on record for $[[288, 12, 18]]$, at $d_X \geq 14$; the ZX-duality permutation — Bravyi et al.'s lemma, not ours — packaged as a $\sim$15 ms checkable certificate; and a trusted base of 649 standard-library Python lines with no solver and no proof assistant in it. | | |

TABLE 2. Claim-by-claim provenance.

*Adjacent work on certificate import.* PBLean [Sze26] imports VeriPB pseudo-Boolean certificates into Lean 4. It is not quantum, and it requires Lean, so it sits on the opposite side of design constraint (ii); but it is the closest prior art we found to the general project of making solver output externally checkable in a small trusted base, and we cite it as such.

1.5. **What is not claimed.** We do not claim any new distance *value*. We do not claim priority for machine-checked quantum distance proofs. We do not claim that $[[288, 12, 18]]$ has $d < 18$; our $d_X \geq 14$ is a lower bound, four short of the literature value, and the gap is a limitation of our encoding, not evidence. We do not certify $k$: the code dimensions in Table 1 are recomputed, not certified, though side condition (c) of Theorem 3.1 pins the logical dimension implicitly, and the audit recomputed every $k$ independently. We do not claim the corpus is free of defects: four are reported in §8 — one fixed in this release, three not — including a genuine latent soundness hole in a checker branch that no shipped certificate exercises. And we do not claim novelty for the framing itself: this is a verification contribution, and the reason to read it is the audit of §7, not the numbers in Table 1.

## 2. CODES, DISTANCES, AND THE TWO CERTIFICATE PROBLEMS

We work over $\mathbb{F}_2$. A binary vector $x \in \mathbb{F}_2^n$ has weight $\mathrm{wt}(x)$ and support $\mathrm{supp}(x)$. For a matrix $M$, $\mathrm{rowsp}(M)$ is its row space, and for a permutation $\pi$ of $\{0, \ldots, n-1\}$ we write $M\pi$ for the column-permuted matrix $(M\pi)_{r,i} = M_{r,\pi(i)}$.

**Definition 2.1** (CSS code)**.** A CSS code is a pair $H_X \in \mathbb{F}_2^{m_X \times n}$, $H_Z \in \mathbb{F}_2^{m_Z \times n}$ with $H_X H_Z^{\mathsf{T}} = 0$. Its dimension is $k = n - \operatorname{rank} H_X - \operatorname{rank} H_Z$. An *$X$-logical* is a vector $x$ with $H_Z x = 0$; it is *nontrivial* if $x \notin \operatorname{rowsp}(H_X)$. Set

$$d_X = \min\{\operatorname{wt}(x) : H_Z x = 0,\ x \notin \operatorname{rowsp}(H_X)\},$$

and $d_Z$ symmetrically with $X$ and $Z$ interchanged. The code distance is $d = \min(d_X, d_Z)$.

The identity $d = \min(d_X, d_Z)$ is the standard CSS fact and we use it without further comment; it is item 5 in the trusted-base inventory of §4. For a general (non-CSS) stabilizer code we use the symplectic form $\omega$ on $\mathbb{F}_2^{2n}$, $\omega(p, q) = \sum_i (p_i q_{n+i} + p_{n+i} q_i)$, a stabilizer matrix $S \in \mathbb{F}_2^{m \times 2n}$ with pairwise-commuting rows, and qubit weight $\operatorname{wt}_q(e) = \#\{i : e_i = 1 \text{ or } e_{n+i} = 1\}$.

Certifying $d$ splits into two problems of completely different character. An *upper* bound is existential and needs only a witness. A *lower* bound is universal: it asserts that a search space of size $\binom{n}{\le K}$ intersected with a coset is empty, and no small object witnesses that directly. The design question this note answers is what the smallest honest replayable object for the second problem looks like.

## 3. The certificate formats

3.1. **Upper bounds: witness pairs.** An $X$-sector upper-bound certificate is a pair $(x, z) \in \mathbb{F}_2^n \times \mathbb{F}_2^n$ together with a declared weight $w$. The checker (`check_witness.py`, 95 lines) verifies

$$H_Z x = 0, \qquad H_X z = 0, \qquad x \cdot z = 1, \qquad \operatorname{wt}(x) = w.$$

Soundness is immediate: $H_Z x = 0$ makes $x$ an $X$-logical; every element of $\operatorname{rowsp}(H_X)$ is orthogonal to $z$ because $H_X z = 0$, so $x \cdot z = 1$ forces $x \notin \operatorname{rowsp}(H_X)$. Hence $d_X \le w$. Three matrix–vector products; no solver, no proof, no search. The gross-code witnesses are 970-byte files and check in 7 ms.

The general stabilizer form is the same shape with $\omega$ in place of the dot product: $(e, f)$ with $\omega(s, e) = \omega(s, f) = 0$ for every stabilizer row $s$, and $\omega(e, f) = 1$, certifies $d \le \operatorname{wt}_q(e)$. The five-qubit code carries this variant end to end.

3.2. **Lower bounds: a CNF the checker writes itself.** Fix a sector, say $X$, and a bound $K$. The canonical CNF $\Phi_K(H_Z, P)$ over variables $x_1, \dots, x_n$ (plus auxiliaries) asserts

(1) $H_Z x = 0$, one Tseitin XOR chain per row of $H_Z$, with the chain output forced false;
(2) $b_j = z_j \cdot x$ for each row $z_j$ of a *pairing* matrix $P$, one XOR chain each, output free;
(3) the clause $(b_1 \vee \cdots \vee b_p)$;
(4) a Sinz sequential at-most-$K$ counter [Sin05] on $x_1, \dots, x_n$;
(5) optionally, forced unit literals (§3.3).

A lower-bound certificate is a JSON file naming $H_X, H_Z, P$, the value $K$, the sector, and for each instance a list of forced literals and an LRAT file. The checker `check_lower.py` (481 lines) does four things: verifies the side conditions of Theorem 3.1; *regenerates* $\Phi_K(H_Z, P)$ from the raw matrices in a fixed variable order; replays the LRAT against its own clause list; and, for symmetry-broken certificates, verifies the hypotheses of Lemma 3.2.

The shipped `.cnf` files are never read. They are informational, and the audit confirmed this by appending a contradictory clause pair to one of them and observing the check pass unchanged (§7).

**Theorem 3.1** (Exactness of the CSS encoding)**.** *Let $H_X \in \mathbb{F}_2^{m_X \times n}$, $H_Z \in \mathbb{F}_2^{m_Z \times n}$ and $P \in \mathbb{F}_2^{p \times n}$ satisfy*

(a) $H_X H_Z^{\mathsf{T}} = 0$;
(b) $H_X P^{\mathsf{T}} = 0$;
(c) $\operatorname{rank}\begin{pmatrix} H_Z \\ P \end{pmatrix} = n - \operatorname{rank} H_X$.

*Then*

$$\{x \in \mathbb{F}_2^n : H_Z x = 0,\ Px \neq 0\} \;=\; \ker H_Z \setminus \operatorname{rowsp}(H_X),$$

*the set of nontrivial $X$-logicals. Consequently $\Phi_K(H_Z, P)$ is satisfiable if and only if there is a nontrivial $X$-logical of weight at most $K$; an unsatisfiability proof for $\Phi_K(H_Z, P)$ certifies $d_X \geq K + 1$.*

*Proof.* Put $V = \ker H_Z \cap (\operatorname{rowsp} P)^{\perp} = (\operatorname{rowsp} H_Z + \operatorname{rowsp} P)^{\perp}$. By (a), $\operatorname{rowsp}(H_X) \subseteq \ker H_Z$; by (b), $\operatorname{rowsp}(H_X) \subseteq (\operatorname{rowsp} P)^{\perp}$. Hence $\operatorname{rowsp}(H_X) \subseteq V$. By (c),

$$\dim V \;=\; n - \operatorname{rank}\begin{pmatrix} H_Z \\ P \end{pmatrix} \;=\; \operatorname{rank} H_X \;=\; \dim \operatorname{rowsp}(H_X),$$

so $V = \operatorname{rowsp}(H_X)$. Now for $x \in \ker H_Z$ we have $Px \neq 0$ iff $x \notin (\operatorname{rowsp} P)^{\perp}$ iff $x \notin V = \operatorname{rowsp}(H_X)$, which is the claim. The second sentence follows because clauses (1)–(3) of $\Phi_K$ encode exactly $H_Z x = 0 \wedge Px \neq 0$ and clause group (4) encodes $\operatorname{wt}(x) \leq K$. □

Both directions of Theorem 3.1 are load-bearing and for different reasons. Condition (b) gives *soundness*: without it a satisfying assignment need not be a nontrivial logical, and UNSAT would prove nothing about the code. Condition (c) gives *completeness*: without it the encoding could miss logicals, and UNSAT would prove less than claimed. The checker verifies (a) and (b) by explicit $\mathbb{F}_2$ inner products and (c) by two Gaussian eliminations over bitmask integers, on every certificate, before it looks at the proof. This is why the pairing matrix $P$ — which the pipeline produced, and which a hostile pipeline could try to weaken — is not trusted input. Negative control 7 of §7 deletes one of its twelve rows and the checker rejects on condition (c).

3.3. **Symmetry breaking, and the orbit lemma.** For the largest instances the search space can be cut by the code's own automorphisms. A symmetry-broken certificate ships permutations $\pi_1, \ldots, \pi_r$, a block boundary $b$, and *two* instances with prescribed forced literals.

**Lemma 3.2** (Orbit lemma)**.** *Let $G \leq S_n$ be generated by permutations $\pi$ such that*

(i) *$\pi$ preserves the partition $\{0, \ldots, b-1\} \sqcup \{b, \ldots, n-1\}$;*
(ii) *$\operatorname{rowsp}(H_X \pi) = \operatorname{rowsp}(H_X)$ and $\operatorname{rowsp}(H_Z \pi) = \operatorname{rowsp}(H_Z)$;*

*and suppose $G$ acts transitively on each of the two blocks. If a nontrivial $X$-logical of weight $\leq K$ exists, then one exists whose support either contains qubit 0, or is disjoint from $\{0, \ldots, b-1\}$ and contains qubit $b$.*

*Proof.* By (ii), $x \mapsto x \circ \pi$ maps $\ker H_Z$ to itself and $\operatorname{rowsp}(H_X)$ to itself, hence maps nontrivial $X$-logicals to nontrivial $X$-logicals, and it preserves weight. Let $x$ be nontrivial of weight $\leq K$. If $\operatorname{supp}(x)$ meets the left block, pick $i$ in the intersection and $g \in G$ with $g(0) = i$, possible by transitivity; then $y = x \circ g$ has $y_0 = 1$. Otherwise $\operatorname{supp}(x)$ meets the right block, say at $i \geq b$; pick $g \in G$ with $g(b) = i$; then $y = x \circ g$ has $y_b = 1$, and $y$ still vanishes on the left block because $g$ preserves blocks. □

The two cases are exactly the two forced-literal patterns the checker demands: instance 0 has `forced` $= [1]$ and instance 1 has `forced` $= [-1, \ldots, -b, b+1]$. Everything Lemma 3.2 hypothesizes is machine-checked: that each shipped array is a permutation, that it preserves the blocks (index comparison), that it is a rowspace automorphism of both $H_X$ and $H_Z$ (rank

identity $\operatorname{rank}(H) = \operatorname{rank}(H \cup H\pi)$), that the generated group is block-transitive (breadth-first search over the generators and their inverses), and that the two forced patterns are literally the two above. Only the four-line implication in the proof is human-verified. For $[[144, 12, 12]]$ and $[[288, 12, 18]]$ the two shipped permutations were independently identified in the audit as the torus translations $x^1y^0$ and $x^0y^1$.

*Remark* 3.3. The gross code does not need this lemma. The 868 MB $X$ proof and the 672 MB $Z$ proof are single-instance certificates with empty `forced` lists, and the checker's non-symmetry branch asserts exactly that: one instance, no forced literals. So $d([[144, 12, 12]]) = 12$ is certified with the orbit lemma removed from the trusted base. The symmetry-broken $X$ certificate is retained because it is 7× smaller and 17× faster to replay, and because it demonstrates the mechanism at $n = 144$ before it is used in earnest at $n = 288$.

3.4. **The ZX-duality certificate.** For BB codes $d_X = d_Z$. This is not our observation: it is a lemma in the supplemental material of Bravyi et al. [BCGMRY], invoked there in exactly the way we invoke it — to halve the work — with the remark that the hypothesis in question "is satisfied for BB LDPC codes". What we add is an artifact: an explicit permutation, shipped as a file, that turns the lemma into something a reader checks in about 15 ms instead of something a reader believes.

**Lemma 3.4** (Duality)**.** *Let* $\Pi$ *be a permutation of the* $n$ *qubits with* $\operatorname{rowsp}(H_X\Pi) = \operatorname{rowsp}(H_Z)$ *and* $\operatorname{rowsp}(H_Z\Pi) = \operatorname{rowsp}(H_X)$*. Then* $x \mapsto y$*,* $y_i = x_{\Pi(i)}$*, is a weight-preserving bijection from nontrivial* $X$*-logicals to nontrivial* $Z$*-logicals, and* $d_X = d_Z$*.*

*Proof.* Applying $\Pi^{-1}$ to the two hypotheses gives $\operatorname{rowsp}(H_X) = \operatorname{rowsp}(H_Z\Pi^{-1})$ and $\operatorname{rowsp}(H_Z) = \operatorname{rowsp}(H_X\Pi^{-1})$. Let $h$ be a row of $H_X$. Then $h \cdot y = \sum_i h_i x_{\Pi(i)} = \sum_j h_{\Pi^{-1}(j)} x_j = (h \circ \Pi^{-1}) \cdot x$, and $h \circ \Pi^{-1} \in \operatorname{rowsp}(H_X\Pi^{-1}) = \operatorname{rowsp}(H_Z)$, so $h \cdot y = 0$ whenever $H_Z x = 0$. Thus $H_X y = 0$. If $y \in \operatorname{rowsp}(H_Z)$, say $y = cH_Z$, then $x_j = y_{\Pi^{-1}(j)}$ exhibits $x \in \operatorname{rowsp}(H_Z\Pi^{-1}) = \operatorname{rowsp}(H_X)$, contradicting nontriviality of $x$. Weight is preserved because $\Pi$ permutes coordinates. The same argument with $X$ and $Z$ interchanged gives the inverse map, so the correspondence is a bijection and $d_X = d_Z$. □

The certificate is the permutation $\Pi$; the checker (`check_duality.py`, 73 lines) does two rank comparisons. The corpus ships one for each of the five BB codes, and each checks in a few milliseconds to a few tens of milliseconds — 3.8, 4.7, 6.1, 9.9 and 37.9 ms for $n = 72, 90, 108, 144, 288$ on a loaded machine. In every case $\Pi$ is the composite of the block swap with $(a, b) \mapsto (-a, -b)$ on the torus indices.

*Remark* 3.5 (The $n = 288$ certificate, and when it appeared). The audit of §7 found no duality certificate for $[[288, 12, 18]]$ (discrepancy D2 of §8): the generator listed the code but had not been re-run, so what the corpus certified there was $14 \le d_X \le 18$ rather than $14 \le d \le 18$. The auditor independently confirmed that the same permutation family does satisfy both rowspace identities at $n = 288$, so the mathematics was never in doubt — only the artifact was missing, which is exactly the condition this paper exists to avoid. The generator has since been re-run and `bb288/duality.json` now ships. It is the one certificate in the release that is *not* among the audit's 47 checks and not among the 182 hashes in `manifest.json`, both of which predate it; it passes `check_duality.py`, and its permutation was verified a second time, from scratch, against independently loaded matrices. With it, $14 \le d \le 18$ is certified rather than merely true, and Table 1 is worded accordingly. No $Z$-sector witness is needed: $d = \min(d_X, d_Z) = d_X$ once duality is in hand.

3.5. **The symplectic variant.** For a general stabilizer code the encoding uses $3n$ variables — $u_i$, $v_i$ for the $X$- and $Z$-components and $q_i$ for "qubit $i$ is in the support", tied by $(\neg u_i \vee q_i)$, $(\neg v_i \vee q_i)$, $(u_i \vee v_i \vee \neg q_i)$ — with XOR chains for the symplectic products and the at-most-$K$ counter on the $q$'s.

**Proposition 3.6.** *Let $S \in \mathbb{F}_2^{m\times 2n}$ have pairwise $\omega$-orthogonal rows, let $L \in \mathbb{F}_2^{\ell\times 2n}$ satisfy $\omega(s,\lambda) = 0$ for all rows $s$ of $S$ and $\lambda$ of $L$, and suppose $\operatorname{rank}\binom{S}{L} = \operatorname{rank} S + \ell = n + k$ where $k = n - \operatorname{rank} S$. Then $\{e : \omega(s,e) = 0\ \forall s,\ \exists\lambda\ \omega(e,\lambda) = 1\}$ is exactly the set of nontrivial logical operators.*

*Proof.* Let $C$ be the $\omega$-centralizer of $\operatorname{rowsp}(S)$, of dimension $2n - \operatorname{rank} S = n + k$. The radical of $\omega|_C$ is $\operatorname{rowsp}(S)$. The rows of $S$ and $L$ all lie in $C$ and span a subspace of dimension $\operatorname{rank} S + \ell = n + k$, so they span $C$. If $e \in C$ and $\omega(e,\lambda) = 0$ for every row of $L$, then $\omega(e,\cdot)$ vanishes on $C$, so $e$ lies in the radical, i.e. $e \in \operatorname{rowsp}(S)$. The converse inclusion is immediate. □

The five-qubit code $[[5,1,3]]$ carries this path end to end: a 128-byte witness and a 1,514-byte LRAT for $d \geq 3$. It exists to keep the non-CSS branch of the checker exercised; see also defect D4 in §8, which is a missing guard in exactly that branch.

## 4. The trusted base

A certificate is only as good as the list of things its reader must still believe. Here is that list, split into what is re-verified per certificate by code one can read (CHECKED) and what is argued in prose and verified nowhere (ASSUMED).

4.1. **Software.** Three files, 649 lines of Python in total: `check_witness.py` (95), `check_duality.py` (73), `check_lower.py` (481). Their only imports are `gzip`, `json`, `os`, `subprocess`, `sys`, `tempfile` and `time`. No `numpy`, no compiled helper, no network. CPython and the operating system must be trusted. If the optional `--external` path is used to delegate LRAT replay to a compiled checker, that binary and the temporary-file marshalling re-enter the trusted base; the audit of §7 never used it, so for that audit the trusted base contained no compiled code at all.

*Not* trusted, and demonstrably so: CaDiCaL; the generating pipeline (`certify.py`, `qec_lib.py`, `gen_duality.py`, `manifest.py`); the shipped `.cnf` files; `meta.json`; `manifest.json`; and the prose of this paper. All of them can be deleted and every certificate still verifies.

*Remark* 4.1 (The checker moved after the audit). The auditor of §7 read a 419-line `check_lower.py`; the file shipped with this release is 481 lines. The difference is one addition: an optional truncated Bailleux–Boufkhad totalizer as an alternative to the Sinz cardinality encoding, put in for larger $[[288,12,18]]$ rungs that are still running and are not part of this release. It is selected only by a certificate that declares `"cardinality": "totalizer"`, and *no certificate in this release declares it*: all take the Sinz default, which is the code path the audit read and exercised. A reader minimising trusted-base surface can delete the totalizer branch and re-run everything. We flag the drift rather than quietly re-using the audit's line count, because the size of that number is one of the claims.

4.2. **Machine-checked, per certificate.**

- CSS orthogonality $H_X H_Z^{\mathsf{T}} = 0$ — condition (a).
- Every pairing row in the kernel of the quotient matrix — condition (b), soundness.
- The rank/spanning identity — condition (c), completeness.

- For the symplectic branch: commutativity of $S$, $L$ in the centralizer, and the two rank identities of Proposition 3.6.
- For `_sym` certificates: that each permutation is a permutation, preserves blocks, is a rowspace automorphism of both $H_X$ and $H_Z$; that the generated group is block-transitive; and that the forced patterns are exactly the two Lemma 3.2 licenses.
- For duality certificates: both rowspace identities.
- Regeneration of the entire CNF from the raw matrices, and RUP replay of the LRAT against *that* clause list.

### 4.3. **Assumed.**

(1) *The Sinz sequential at-most-K counter is complete* [Sin05]: every weight-$\leq K$ assignment extends to a satisfying assignment of the counter variables. If it were accidentally over-constraining, UNSAT would mean less than claimed. The emitted clause set was read against [Sin05] and is the standard one.
(2) *The Tseitin XOR gate is definitional* [Tse68]: the four clauses per gate encode $u = a \oplus b$ and are always extendable.
(3) *The orbit lemma*, Lemma 3.2 — four lines, proved above, machine-checked hypotheses, human-verified implication. Used by the symmetry-broken $[[144, 12, 12]]$ $X$ certificate and all three $[[288, 12, 18]]$ rungs. **Not used by** the symmetry-free $[[144, 12, 12]]$ certificates (Remark 3.3).
(4) *The duality lemma*, Lemma 3.4 — likewise, checked hypotheses, human-verified implication.
(5) *The CSS fact* $d = \min(d_X, d_Z)$, used to turn a certified pair into a statement about the code. No script touches it.
(6) *LRAT semantics.* The internal checker implements RUP-with-hints: negate the lemma, unit-propagate through the hinted clauses in order, demand a conflict. It skips hints that are already satisfied or non-unit — sound, since skipping can only fail to find a conflict, never invent one — and it *refuses* negative (RAT) hints outright, which incidentally establishes that every proof in the corpus is pure RUP. It requires an empty clause to be derived and verified.
(7) *That the matrices are the intended code.* No certificate establishes this; the checkers take `HX.txt` and `HZ.txt` at face value. Section 7.1 closes this for the five BB codes at byte level.

In one sentence: a skeptic must believe that three short standard-library Python files do what they appear to do, that CPython and the OS are not lying, that the Sinz and Tseitin encodings are standard and complete, that the four-line duality lemma and the four-line orbit lemma are correct, and that $d = \min(d_X, d_Z)$. Everything else can be thrown away.

## 5. The gross code

$[[144, 12, 12]]$ is the $\ell = 12$, $m = 6$ member of the BB family, with $A = x^3 + y + y^2$ and $B = y^3 + x + x^2$, $H_X = [A \mid B]$, $H_Z = [B^\mathsf{T} \mid A^\mathsf{T}]$ [BCGMRY]. Its certified distance decomposes as follows; all byte counts are the shipped files.

| statement | artifact | bytes | solver (s) | replay |
|---|---|---|---|---|
| $d_X \le 12$ | `witness_X.json` | 970 | 0.025 | 7.4 ms |
| $d_Z \le 12$ | `witness_Z.json` | 970 | 0.029 | 7.3 ms |
| $d_X \ge 12$, symmetry-broken | `lower_X_K11_sym.json` +2 LRATs | 123,857,070 | 45.0 | 10.1 s |
| $d_Z \ge 12$, symmetry-free | `lower_Z_K11.json` +1 LRAT | 671,988,205 | 227.3 | 72.9 s |
| $d_X \ge 12$, symmetry-free | `lower_X_K11.json` +1 LRAT | 867,803,294 | 342.3 | 176.4 s |
| $d_X = d_Z$ | `duality.json` | 103 + 466 | — | 9.9 ms |

Total solver time for the complete symmetry-free route is 569.6 s: under ten minutes on a laptop. Total replay time for that route, in pure Python with no compiled code anywhere, is 249.3 s. The two symmetry-free proofs are what let us say that $d([[144, 12, 12]]) = 12$ holds on the trusted base of §4 *minus* item 3.

The value $d = 12$ is not new; it is [BCGMRY, Table 3], obtained by mixed integer programming [LAR11], confirmed exactly at MIP gap zero by [CCKF26], and reproduced by SAT in [CJL26]. A machine-checked proof of it is not new either: the LEAN-QEC repository reports one as of 2026-07-10 (§1.4). What is offered here is the shape of the artifact — an 868 MB file plus a 481-line reader, with no symmetry hypothesis, no solver and no proof assistant — and the fact that it replays inside 51 MB of memory.

*Remark* 5.1 (Timing variability). The replay times above are the audit's measurements on an otherwise idle machine. A re-run of the symmetry-broken $X$ certificate performed while writing this note, under load, took 30.4 s rather than 10.1 s. All timing figures in this paper should be read with a factor of three of slack; the byte counts and the pass/fail outcomes are exact.

## 6. [[288, 12, 18]]

For the $\ell = m = 12$ member, $A = x^3 + y^2 + y^7$, $B = y^3 + x + x^2$, we certify a ladder in the $X$ sector, each rung an independently valid symmetry-broken certificate:

$$d_X \ge 10 \text{ (65 MB, 11.0 s solver)}, \quad d_X \ge 12 \text{ (351 MB, 85.6 s)}, \quad d_X \ge 14 \text{ (2.94 GB, 512.6 s)},$$

together with a weight-18 witness for $d_X \le 18$ and a duality certificate (Remark 3.5) that converts both into statements about $d$. The $K = 13$ proof is 2,941,958,076 + 191,479 bytes across the two orbit instances and replays in pure Python in 413.6 s with a peak resident set of 79 MB — the memory figure being the interesting one, since the proof is more than thirty times larger than the memory used to check it.

*Remark* 6.1 (What the literature says, stated correctly). An earlier internal draft of our results claimed that Bravyi et al. screened this code with a heuristic and conceded that its $d \le 18$ "is unlikely to be tight". That is a misreading and we correct it here. In [BCGMRY], the quoted caveat concerns the *circuit-level* distance $d_{\text{circ}} \le 18$, a different quantity from the code distance. Table 3 of that paper lists [[288, 12, 18]] with no "≤", while [[360, 12, ≤ 24]] and [[756, 16, ≤ 34]] carry one, and the caption states that the notation "≤ $d$" marks entries for which only an upper bound is known; the supplemental material states that the actual distance "of each candidate code was computed using the integer linear programming method". So [BCGMRY] asserts $d = 18$ exactly for this code, by ILP and without a checkable artifact. Our interval $[14, 18]$ is therefore *not* new information about the value, and any suggestion that it is the first distance information produced for this code — a suggestion the earlier draft made — is withdrawn. What $d_X \ge 14$ is, is the strongest lower bound for this code that anyone can check.

The strongest quantity previously published for this code as a *lower bound* is $d \ge 11$, from Chen, Jafari and Lai [CJL26], obtained by several solvers under 7200 s timeouts and asserted

rather than certified; their public repository `guluchen/QDistSAT` contains no proof artifacts. Our $d_X \geq 14$ improves on it and adds what neither it nor [BCGMRY] has: a file. As far as the sweep recorded in §1.4 could determine, it is the only machine-checkable lower bound on record for $[[288, 12, 18]]$ at any strength.

Where the wall is, concretely: proof size grows roughly 8× per ladder rung on this encoding, so $K = 15$ is an overnight job at an estimated 25 GB, and $K = 17$ — the rung that would close the gap to 18 — is out of reach of this encoding on this machine. Better encodings, in particular the location-indexed encoding of [ELWT26], proof compression, and per-rung parallelism are the obvious next moves, and none of them is ours.

## 7. Independent verification

The corpus was re-checked by a separate agent instance with no access to the pipeline and no shared code, on the same machine, using `/usr/bin/python3` (CPython 3.9.6). Its report is `INDEPENDENT-VERIFICATION.md`. Summary: 47 checks run, 47 passed, 0 failed; 5,459,315,046 bytes (5.08 GiB) of LRAT replayed in approximately 746 s of wall clock; peak resident set 79 MB; all 182 SHA-256 entries in `manifest.json` re-hashed and matching. Because `lrat-check` is not present in the repository (defect D3), *every* proof — including the 2.94 GB one — was replayed with the internal pure-Python checker. That is the stronger result. Eleven negative controls, built by the auditor on scratch copies, were all correctly rejected (§7.3), and six codes were cross-checked by brute force (§7.2). This audit, not the distance values, is the load-bearing part of the paper.

7.1. **Are these the right matrices?** This is the check the certificates cannot perform on themselves, and it was done first. The auditor rebuilt $H_X$ and $H_Z$ for all five BB codes directly from the construction in [BCGMRY] — $n = 2\ell m$, $x = S_\ell \otimes I_m$, $y = I_\ell \otimes S_m$ over $\mathbb{F}_2$ with $S_r$ the cyclic shift, $H_X = [A \mid B]$, $H_Z = [B^\mathsf{T} \mid A^\mathsf{T}]$ — using that paper's Table 3 parameters. For every one of `bb72`, `bb90`, `bb108`, `bb144`, `bb288`, the shipped `HX.txt` and `HZ.txt` are **byte-identical** to the reconstruction: no row operations, no column permutation, no block swap. Independently recomputed $k = n - \operatorname{rank} H_X - \operatorname{rank} H_Z$ gives $12, 8, 8, 12, 12$, matching the published parameters, with row weight 6 and column weight 3 throughout. Every algebraic side condition was then re-run a second time, in the auditor's own implementation, against the *reconstructed* matrices rather than the shipped ones; all agreed.

7.2. **Brute force where brute force is possible.** For the six codes small enough, the auditor computed the minimum distance by exhaustive Gray-code enumeration over a kernel basis, with code sharing nothing with either the pipeline or the checkers: Steane, surface $d = 3$, surface $d = 5$, Golay, the five-qubit code, and surface $d = 7$ (a $2^{25}$ search, 242 s). Six exact distances, six agreements with the certified values. From $[[72, 12, 6]]$ upward the search space is $2^{42}$ and beyond, which is the entire point of the certificates.

7.3. **Negative controls.** Eleven tampered artifacts were built on scratch copies and all eleven were rejected, each with the specific error one would want: a flipped witness bit ("witness does not commute"), an understated weight, a relabelled $K$ ("LRAT lemma 2015 NOT verified"), a deleted empty clause, a truncated proof, a live-but-wrong hint id, a gutted pairing matrix (rejected on condition (c), "31 $\neq$ 42"), a flipped bit of $H_X$ (rejected on condition (a)), a swapped sector label, a corrupted hint id, and an identity duality permutation. In addition, a contradictory clause pair was appended to `bb72/lower_X_K5.cnf` and the check passed unchanged and in the same time — a positive control on the claim that the shipped CNF is genuinely never read.

## 8. Defects found

Reported verbatim, because a paper about trusted bases that suppresses its own audit findings is not one.

**D1 (documentation).** The results file cites a `tamper_test/` directory that does not exist in the repository. The claim is true — the controls were rebuilt from scratch and all rejected (§7.3) — but the citation is dangling.

**D2 (real, touched a headline, now closed).** At audit time there was no duality certificate and no $Z$-witness for $[[288, 12, 18]]$, so the corpus certified $14 \le d_X \le 18$, not $14 \le d \le 18$: the generator listed that code and had evidently not been re-run. The underlying mathematics was verified independently by the auditor, so the claim was true and uncertified, which is precisely the condition this paper exists to avoid. This is the one defect of the four that is fixed in this release: the generator has been re-run, `bb288/duality.json` ships, it passes `check_duality.py`, and its permutation has been re-verified from scratch. The caveats are that this certificate postdates the audit and is therefore outside its 47 checks, and that `manifest.json` — whose 182 entries the audit re-hashed — has deliberately not been regenerated, so it does not list the new file. See Remark 3.5.

**D3 (documentation).** `run_all.sh` requires `tools-drat-trim/lrat-check`, which is absent with no source or URL, so the `lrat-check` timings previously quoted are not reproducible from the repository as shipped. The pure-Python path is reproducible and is what Table 1 reports. Two earlier pure-Python figures were also stale in the conservative direction (21 s quoted versus 6.2 s measured for $[[108, 8, 10]]$; 17.3 s versus 10.1 s for the symmetry-broken $[[144, 12, 12]]$ certificate), and the quoted throughput of 3.4 MB/s is closer to 12 MB/s.

**D4 (latent soundness hole).** In the CSS branch, `check_lower.py` requires a certificate with no symmetry block to have exactly one instance with an empty forced list. The symplectic branch has *no such guard*: it passes the certificate's forced literals straight into the encoder, which emits them as unit clauses. The auditor demonstrated this by adding `"forced": [-1,-2,-3]` to the five-qubit certificate and obtaining `OK lower bound: d >= 3` from a proof that now covers only assignments avoiding those qubits. No shipped certificate is affected — the five-qubit certificate's forced list is empty, and this was checked — but the checker would accept a forged one. The one-line fix is to mirror the CSS assertion. Two lesser observations in the same vein: the symmetry branch does not assert that there are exactly two instances (harmless, since each is replayed independently), and the CSS branch does not validate the sector label (a typo silently selects the other branch, whose side conditions then fail, as negative control 9 shows).

D4 is the interesting one. It is exactly the class of defect that the certificate discipline is supposed to make findable: a hole in 481 lines of readable Python, found by reading them, rather than a hole in a solver.

## 9. Relation to other work

*Heuristic upper bounds.* QDistRnd [PSKK22] produces upper bounds with no performance guarantee; Stim's logical-error search [Gid21] is documented as heuristic. Both are excellent for what they are. Our upper bounds differ only in shipping the witness.

*Exact methods without artifacts.* The mixed-integer-programming method of Landahl, Anderson and Rice [LAR11] is what [BCGMRY] used and remains the reference for the BB distances; Cruz-Benito, Cross, Kremer and Faro [CCKF26] re-close the gross-code MILP to gap zero. Chen, Jafari and Lai [CJL26] push SAT and MaxSAT to this family at scale and

explicitly do not log proofs. Webster, Jacob and Higgott [WJH26] survey the field and distinguish exact from heuristic methods carefully; their desiderata do not include exportable certificates, and an exhaustive search of their text turns up a single occurrence of certification, referring to internal optimality rather than to an artifact. None of these produces something a third party can replay. That gap, not the numbers, is what this note addresses.

*Proof-assistant approaches.* LEAN-QEC [ELWT26] is the closest work and the one we have most to learn from: their location-indexed encoding is better than ours, and we did not need it only because we stopped at $n = 288$. The design difference is where the trust sits. Their artifact is replayable inside the Lean 4 kernel, which is a stronger guarantee than ours about the *checker*, at the cost of requiring Lean and Mathlib — a toolchain fetch — to obtain it. Ours is replayable by any CPython, which is a weaker guarantee about the checker and a much smaller barrier to a skeptic. Neither dominates. PBLean [Sze26] imports VeriPB certificates into Lean 4 and is the nearest general prior art for the certificate-import problem.

*What we could not find.* A search of the public record on 2026-08-04 — arXiv, code search for LRAT together with quantum distance, and the issue trackers of Stim, `panqec`, `ldpc` and `qLDPC` — turned up no standalone quantum distance certificate requiring neither a solver nor a proof assistant to check. A negative cannot be proved this way; we record the scope of the search rather than a claim.

## 10. Reproducibility

The artifacts sit beside this note in `qec/`. To re-check anything:

```
gunzip certificates/bb144/*.lrat.gz
python3 check_witness.py  certificates/bb144/witness_X.json
python3 check_lower.py    certificates/bb144/lower_X_K11.json
python3 check_lower.py    certificates/bb144/lower_Z_K11.json
python3 check_duality.py  certificates/bb144/duality.json
python3 check_lower.py    certificates/bb288/lower_X_K13_sym.json
```

No virtual environment, no packages, no compiled binary. Any CPython 3.8 or later should do; the audit used the macOS system interpreter, 3.9.6. Expect 176 s, 73 s and 414 s respectively for the three large replays on an idle M4, more under load. `manifest.json` carries the SHA-256 and byte count of all 182 artifacts and `manifest.py` re-checks them.

*What the public repository can and cannot hold.* Four of the proofs — the two symmetry-free $[[144, 12, 12]]$ certificates and the first instance of each of the $K = 11$ and $K = 13$ rungs at $n = 288$ — are between 79 MB and 646 MB compressed and are not carried in git. Everything else is: the descriptors, the parity-check matrices, the pairing and permutation files, the witnesses, the duality certificates, the CNF inputs, the manifest, and every proof small enough to ship, up to and including the 30 MB symmetry-broken $[[144, 12, 12]]$ certificate and the $K = 9$ rung at $n = 288$. So a reader who clones and runs nothing but CPython can still replay a certified $d = 3$ for all four sanity-tier codes, $d = 7$ for Golay and the $d$=7 surface code, $d = 6$ for $[[72, 12, 6]]$, $d = 10$ for $[[90, 8, 10]]$ and $[[108, 8, 10]]$, $d = 12$ for the gross code — by the symmetry-broken $X$ certificate together with the duality certificate and the weight-12 witness, the two symmetry-free proofs being exactly the ones that do not fit — and $d_X \geq 10$ at $n = 288$. For the four omitted proofs the release ships `REGENERATE.md`: the exact CaDiCaL invocation for each, with the expected byte count and the SHA-256 the result must have. Regenerating them puts a solver back in the loop for the *production* of the proof, which is where a solver has always been allowed to sit; the check that follows still does not trust it. The generating pipeline (`certify.py`, `qec_lib.py`, `run_all.sh`) is included for completeness and

is not needed to check anything; it requires `numpy` and a CaDiCaL binary, and as noted in D3 its full `run_all.sh` path additionally expects an `lrat-check` binary that is not vendored.

## Acknowledgments

The debt to Bravyi, Cross, Gambetta, Maslov, Rall and Yoder is total: the codes are theirs, the distances are theirs, and the $d_X = d_Z$ lemma that halves the work at every BB instance is theirs. To Landahl, Anderson and Rice for the integer-programming distance method that produced the reference values, and to Cruz-Benito, Cross, Kremer and Faro for closing the gross-code MILP to gap zero and thereby fixing the value we certify. To the LEAN-QEC authors for putting kernel-checked quantum distance proofs on the map, for naming the gross code as their target in print, and for reaching it in their repository before we finished writing; several claims in an earlier draft of this note had to be withdrawn on discovering that, and the discovery was entirely to the good. To Chen, Jafari and Lai for the SAT benchmark against which our $n = 288$ bound is measured. To Marijn Heule and coauthors for LRAT and `drat-trim`, without which none of this would have an interchange format, and to Armin Biere and coauthors for CaDiCaL. To Carsten Sinz for the cardinality encoding and to G. S. Tseitin for the gate encoding that the whole construction rests on.

Finding that a result one has produced was produced first by someone else is the ordinary condition of working in a field that is moving faster than the writing; we have tried to record it accurately rather than minimally.

## References

[BCGMRY] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall and T. J. Yoder, *High-threshold and low-overhead fault-tolerant quantum memory*, Nature **627** (2024), 778–782; arXiv:2308.07915.

[Bie20] A. Biere, K. Fazekas, M. Fleury and M. Heisinger, *CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020*, in: Proc. SAT Competition 2020, Dept. of Computer Science Report Series B, Univ. of Helsinki, 51–53. Version used here: CaDiCaL 3.0.1.

[CS96] A. R. Calderbank and P. W. Shor, *Good quantum error-correcting codes exist*, Phys. Rev. A **54** (1996), 1098–1105.

[CJL26] Chen, Jafari and Lai, arXiv:2606.12445, May 29, 2026 (SAT- and MaxSAT-based computation of quantum code distances). Repository `guluchen/QDistSAT`; inspected August 4, 2026, and containing no proof or certificate artifacts.

[CCKF26] J. Cruz-Benito, A. W. Cross, F. Kremer and I. Faro (IBM Quantum), arXiv:2606.02418, June 1, 2026. Computes the minimum distance of the gross code $[[144, 12, 12]]$ exactly by mixed-integer linear programming, closing the MIP gap to 0; no optimality certificate is exported.

[CFHKS17] L. Cruz-Filipe, M. J. H. Heule, W. A. Hunt Jr., M. Kaufmann and P. Schneider-Kamp, *Efficient certified RAT verification*, in: Automated Deduction — CADE-26, Lecture Notes in Comput. Sci. **10395**, Springer, 2017, 220–236.

[ELWT26] Ehatamm, Lee, Wu and Tao, arXiv:2605.16523v1, May 15, 2026 (the LEAN-QEC system paper: SAT-based quantum code distance proofs replayed in the Lean 4 kernel). Repository `VerifiedQC/Lean-QEC`; statements about the repository in this note refer to commit `c73827d` of July 10, 2026, inspected August 4, 2026. Only v1 of the preprint existed at that date.

[Gid21] C. Gidney, *Stim: a fast stabilizer circuit simulator*, Quantum **5** (2021), 497. The documentation of `search_for_undetectable_logical_errors` states that the method is heuristic.

[Got97] D. Gottesman, *Stabilizer codes and quantum error correction*, Ph.D. thesis, California Institute of Technology, 1997; arXiv:quant-ph/9705052.

[HHW13] M. J. H. Heule, W. A. Hunt Jr. and N. Wetzler, *Trimming while checking clausal proofs*, in: Formal Methods in Computer-Aided Design (FMCAD), 2013, 181–188.

[LAR11] A. J. Landahl, J. T. Anderson and P. R. Rice, *Fault-tolerant quantum computing with color codes*, arXiv:1108.5738, 2011. Source of the mixed-integer-programming distance computation used in [BCGMRY].

[PSKK22] L. P. Pryadko, V. A. Shabashov and V. K. Kozin, *QDistRnd: A GAP package for computing the distance of quantum error-correcting codes*, J. Open Source Softw. **7** (2022), 4120; doi:10.21105/joss.04120. Upper bounds only, with no performance guarantee.

[Sin05] C. Sinz, *Towards an optimal CNF encoding of Boolean cardinality constraints*, in: Principles and Practice of Constraint Programming — CP 2005, Lecture Notes in Comput. Sci. **3709**, Springer, 2005, 827–831.

[Ste96] A. M. Steane, *Error correcting codes in quantum theory*, Phys. Rev. Lett. **77** (1996), 793–797.

[Sze26] S. Szeider et al., arXiv:2602.08692, 2026 (PBLean: importing VeriPB pseudo-Boolean proof certificates into Lean 4).

[Tse68] G. S. Tseitin, *On the complexity of derivation in propositional calculus*, in: Studies in Constructive Mathematics and Mathematical Logic, Part II, 1968, 115–125.

[Var97] A. Vardy, *The intractability of computing the minimum distance of a code*, IEEE Trans. Inform. Theory **43** (1997), 1757–1766.

[WJH26] Webster, Jacob and Higgott, arXiv:2603.22532, March 23, 2026 (a survey and comparison of exact and heuristic methods for quantum code distance).

Independent researcher, Half Ounce Research
*Email address*: daniel@halfounce.io